Helping Hand - M3 Alpha Build Report

Team Members: Gael Garcia, Srinitha Srikanth, Kali Schuchhardt, Brian Paz
GitHub Repository: https://github.com/GG1627/helping-hand

Alpha Build Overview

Helping Hand is a wearable glove intended to help users learn American Sign Language (ASL). The project includes an ESP32, five flex sensors, a six-axis IMU, and a Flutter mobile application that runs on Android. During the alpha build, our main focus was recognizing static letter and number hand poses from finger-bend measurements, providing feedback through the app, and saving learned progress. The alpha connected physical sensing, embedded classification, Bluetooth communication, app feedback, and persistent storage into one learning flow.

Integrated Architecture and Vertical Features

The glove includes five flex sensors, one per finger, which measure finger bend through voltage-divider circuits. The IMU supplied motion telemetry for display and recording, but its measurements were not used by the alpha's static classifier. This motion data also supports preparation for future word recognition.

The ESP32 runs the flex-sensor values through a static MLP classifier and sends the prediction and confidence score through Bluetooth Low Energy (BLE) to the Android phone. The user selects a letter or number in the app and attempts the sign. To mark it learned, the prediction must match the selected target at a confidence of at least 80%, maintained for at least 750 milliseconds and five packets.

This flow connects three main architectural elements:

External Interface: The user interacts with the physical glove and Flutter app. The app provides letter and number selection, connection controls, live feedback, and learned-progress indicators.

Internal Systems: Classification runs directly on the ESP32. The app checks incoming predictions for a stable match before marking the selected letter or number learned.

Persistent State: Progress is saved locally first and then synchronized to Firestore. Learned letters, learned numbers, and completed exercises are associated with an anonymous Firebase identity for that installation. Users could not yet create a named account and sign in on another device to access their progress.

Usability and User Feedback

Interface and Navigation: The app contains a home dashboard showing overall progress, an Alphabet tab with A-Z targets, and a Numbers tab with targets 0-9. BLE Testing provides glove connection controls and displays live data, including raw and normalized flex-sensor values. Record Signs provides controls for recording labeled sensor trials, reviewing them, and saving or exporting the recordings. Practice and progress controls include retry and reset options.

Perception: During practice, users can see the current prediction, confidence score, and connection state. A hold-progress bar shows how long the matching prediction has been maintained. Once a letter or

number is learned, its tile changes to a darker color. The app also displays storage and synchronization status so users can tell whether progress is saved locally, syncing, synced, or experiencing a failure.

Responsiveness: BLE and Firebase operations run asynchronously so the app can remain usable while waiting for responses. Progress is stored locally before cloud synchronization, allowing local progress access when Firebase is unavailable.

Build Quality and Validation

Our validation included physical observations and software checks. Team members wore the glove and signed multiple letters and numbers while connected to the Fairphone. We observed correct predictions for the signs tested and saw completed items change appearance in the app. These observations demonstrate physical operation, but they are not a measured accuracy result across every model class.

The phone also displayed “Synced,” indicating that the app’s progress synchronization operation completed successfully. The corresponding Firestore records were not independently inspected during that physical test.

Software validation recorded for the alpha included clean Flutter analysis, 16 passing Flutter tests, 23 passing backend tests, and successful Android debug APK and ESP32 firmware builds. These checks supported build readiness but did not replace extended physical testing.

For consistency, practice used the same confidence and stable-hold requirements before awarding completion. For robustness, the app included local storage and visible offline or synchronization-failure states. The alpha also included app screens, visual feedback, and a wearable glove assembly, although the interface styling and physical packaging still needed refinement. Sustained BLE performance and longer-term reliability required further testing.

Known Limitations and Remaining Work

One limitation of the alpha build is that flex sensors measure finger bend but do not directly measure finger spacing or individual finger position. For example, U and V can produce similar flex-sensor readings, while the difference is whether the index and middle fingers are together or spread apart. The alpha sensor setup cannot directly detect that difference, which limits reliable recognition of signs with similar finger-bend patterns.

At the alpha milestone, the following work remained:

- Investigate ways to distinguish signs with similar finger bending but different finger positions.
- Complete broader physical validation and sustained BLE testing.
- Add named accounts so users could sign in and access their saved progress.
- Train and integrate a temporal model for word recognition using sensor data over time.
- Integrate glove-mounted haptic feedback; alpha learning feedback was visual.
- Develop the custom PCB and improve sensor attachment and wiring through sewn glove integration.
- Update the as-built hardware schematic and refine the app’s visual styling and consistency.

* The team drafted this report. Codex assisted with organization, grammar, wording, and technical clarification.